MODULE

21

HTTP, Flask & Jinja2

How the web really works, and your first server-rendered web app.

YOU WILL LEARN

HTTP from scratch	`01_http_from_scratch.py`
Flask basics	`02_flask_basics.py`
Templates, forms & sessions	`03_templates_forms_sessions.py`
Interview Q&A	`INTERVIEW.md`

WHY THE WEB

Frameworks hide HTTP, until something breaks

THE PROBLEM

A form submits twice, a cookie isn't sent, CORS blocks your frontend. Without knowing **HTTP** you can only guess.

THE IDEA

Every web framework turns one HTTP request into one HTTP response. Learn that message format once and every framework makes sense.

PICTURE IT HTTP

What happens when you open a page

🌐 Your browser looks up the server's address (**DNS**), opens a connection (**TCP + TLS** for https), sends a text **request**, and gets a text **response**. The framework only does the middle part.



TRY IT HTTP FROM SCRATCH

A real HTTP request is just text

```
request = (
    "GET /orders?page=2 HTTP/1.1\r\n"
    "Host: shop.de\r\n"
    "Cookie: session=abc123\r\n"
    "\r\n"                                  # blank line = end of headers
)
print(request)

response = "HTTP/1.1 200 OK\r\nContent-Type: application/json\r\n\r\n[]"
status_line, rest = response.split("\r\n", 1)
print("status:", status_line.split()[1])
```

OUTPUT

```
GET /orders?page=2 HTTP/1.1

Host: shop.de

Cookie: session=abc123




status: 200
```

1 Line 1: **method, path, version**; then headers, a blank line, the body.

TRY IT FLASK BASICS

Routes, parameters, JSON, status codes

```python
from flask import Flask, jsonify, request

app = Flask(__name__)
BOOKS = {1: {"id": 1, "title": "Fluent Python"}}

@app.get("/books/<int:book_id>")
def get_book(book_id):
    book = BOOKS.get(book_id)
    if book is None:
        return jsonify(error="not found"), 404
    return jsonify(book)

@app.post("/books")
def create_book():
    data = request.get_json()
    new = {"id": len(BOOKS) + 1, "title": data["title"]}
    BOOKS[new["id"]] = new
    return jsonify(new), 201

c = app.test_client()
print(c.get("/books/1").get_json())
print(c.get("/books/9").status_code)
print(c.post("/books", json={"title": "Python Pro"}).get_json())
```

OUTPUT

```
{'id': 1, 'title': 'Fluent Python'}
404
{'id': 2, 'title': 'Python Pro'}
```

1. **<int:book_id>** converts and validates the path.
2. Return **(body, status)**.
3. **test_client()** calls the app without a server.

PICTURE IT TEMPLATES

Server-side rendering with Jinja2

🎨 A template is an HTML page with **holes**. The view fills the holes with data; Jinja2 **escapes** it so user text can't inject scripts. A **base** layout is reused by every page.



COMMON MISTAKE FORMS

The double-submit bug

✗ WRONG

```python
@app.post("/order")
def order():
    save_order(request.form)
    return render_template("thanks.html")
# user presses F5 -> the browser POSTs again
```

Refreshing a page that was the answer to a POST **repeats the POST**: a second order, a second payment.

✓ RIGHT

```python
@app.post("/order")
def order():
    save_order(request.form)
    flash("Order placed!")
    return redirect(url_for("thanks"))   # 303
# refresh now repeats a harmless GET
```

Post/Redirect/Get: after a successful POST, redirect. Add CSRF tokens to forms (Flask-WTF).

STEP BY STEP COOKIES AND SESSIONS

How a site remembers you

PIECE	WHAT IT DOES	WATCH OUT
`Cookie`	small data the browser sends back on every request	set `HttpOnly`, `Secure`, `SameSite`
`Flask session`	a **signed** cookie (readable, not editable)	never store secrets in it; set a strong SECRET_KEY
`Server-side session`	cookie holds only an ID; data in Redis/DB	can be revoked; scales with a shared store
`Token (JWT)`	sent in the Authorization header by API clients	Module 24

> HTTP is **stateless:** cookies and tokens are how state is added on top.

IN REAL JOBS FLASK

Where Flask fits at work

Small services and internal tools

```
@app.get("/health")
def health(): return {"status": "ok"}
```

Blueprints for bigger apps

```
bp = Blueprint("orders", __name__)
app.register_blueprint(bp, url_prefix="/orders")
```

Production server

```
gunicorn "app:create_app()" -w 4
```

Choose the tool

```
Flask: small + flexible  | FastAPI: typed APIs
Django: batteries included (admin, ORM, auth)
```

INTERVIEW SAY THIS IN THE INTERVIEW

“What happens when you type a URL and press Enter?”

- The browser parses the URL and checks its cache.
- **DNS** resolves the host name to an IP address.
- It opens a **TCP** connection and does the **TLS** handshake for https.
- It sends an **HTTP request** (method, path, headers, cookies).
- The server (load balancer → app server → framework → your view → DB) builds the **response**.
- The browser renders the HTML and fetches CSS, JS and images with more requests.

> **Follow-up they ask next:** ”What’s the difference between a cookie and a session?” (the cookie lives in the browser; a session is server-side state usually found via an ID stored in a cookie).

INTERVIEW SAY THIS IN THE INTERVIEW

“Flask vs Django vs FastAPI?”

- **Flask**: a micro-framework. Minimal core, you choose the extensions. Great for small services.
- **Django**: batteries included: ORM, migrations, admin, auth, forms. Great for full web apps and CRUD-heavy products.
- **FastAPI**: async, type-hint driven, automatic validation and OpenAPI docs. Great for JSON APIs and microservices.
- All are WSGI/ASGI apps; the HTTP fundamentals are identical.

> **Follow-up they ask next:** ”What are WSGI and ASGI?” (the interfaces between Python apps and servers: WSGI is synchronous (gunicorn), ASGI adds async and WebSockets (uvicorn)).

RECAP REMEMBER THESE

Module 21 in 5 lines

1. HTTP = text request (method, path, headers, body) → text response (status, headers, body).
2. Flask: **@app.get/post**, path converters, **jsonify**, (body, status).
3. Jinja2 templates: **extends**, loops, and **auto-escaping** against XSS.
4. After a POST: **redirect** (Post/Redirect/Get); protect forms with CSRF tokens.
5. Cookies carry state; set HttpOnly, Secure, SameSite.

PRACTICE NOW

Run 02_flask_basics.py, then add a DELETE /books/<id> route that returns 204.